

Começando o jogo: atributos e ações básicas

Vida, ataque, defesa e poções — as peças que toda batalha vai usar daqui pra frente.

Tudo que vocês fizeram até aqui — Personagem, Guerreiro, Mago, Inimigo, o esqueleto de batalha — foi treino. Serviu pra pegar prática com `__init__`, métodos, `if/elif/else`, `while`, `for`. Agora o **código do jogo de verdade começa do zero**: classes novas, arquivo novo, sem colar em cima do que já existia. O conhecimento continua todo valendo — só o código recomeça limpo.

Hoje, essas classes novas ganham um conjunto de stats mais completo e três ações de verdade: **atacar**, **defender** e **usar poção**. Ainda sem laço de jogo automático, sem `input()`, sem `random()` — isso vem depois. Hoje é só sobre montar as peças certas, uma de cada vez, e testar elas na mão. Andar pelo mapa e outras mecânicas também ficam pra mais pra frente.

Vocês escolhem os personagens. Guerreiro e Mago são só sugestão — se quiserem inventar as suas próprias classes (Arqueiro, Curandeiro, Ladrão, Bardo, Vampiro, o que for), pode ser. A única regra é manter os mesmos cinco atributos do exemplo abaixo, pra tudo continuar funcionando igual.

Referência rápida — antes de começar

Esse bloco não é exercício. É o padrão que os exercícios abaixo vão pedir pra reaproveitar.

```
class Personagem:
    def __init__(self, nome, vida, ataque, defesa, pocoes):
        self.nome = nome
        self.vida = vida
        self.ataque = ataque
        self.defesa = defesa
        self.pocoes = pocoes

    def receber_dano(self, quantidade):
        dano_real = quantidade - self.defesa
        if dano_real < 1:
            dano_real = 1
        self.vida -= dano_real
        if self.vida < 0:
            self.vida = 0
        print(self.nome, "recebeu", dano_real, "de dano.")
```

Dois detalhes valem a pena notar: primeiro, `defesa` só entra *dentro* de `receber_dano` — ninguém de fora precisa saber que a defesa existe, só o método que calcula o dano. Segundo, o `if dano_real < 1: dano_real = 1` existe por uma razão de design, não só de código: sem esse limite, uma defesa alta o suficiente deixaria o personagem completamente imune, e a batalha nunca terminaria. Todo ataque causa pelo menos 1 de dano, não importa a defesa.

EXERCÍCIOS

Montando as ações, uma de cada vez

Os quatro primeiros constroem as peças. O último testa todas juntas, na mão, sem laço.

Exercício 1

DUAS CLASSES, DO ZERO

Comecem um arquivo novo pro jogo de verdade (por exemplo, um arquivo por classe, dentro de uma pasta própria, separada dos exercícios de treino). Criem **duas classes** nesse arquivo novo — Guerreiro e Mago, como no resto deste PDF, **ou duas classes inventadas por vocês** (Arqueiro, Curandeiro, Ladrão, Bardo, Vampiro — o que fizer sentido pro jogo de vocês). Cada uma com os cinco atributos do exemplo acima — vida, ataque, defesa, pocoes, além do nome. Copiem também o método `receber_dano` pras duas classes. Não precisa (e não deve) reaproveitar nenhum arquivo antigo — é código novo, mesmo que pareça repetido.

REQUISITOS MÍNIMOS

- Duas classes escritas num arquivo novo, cada uma com o seu próprio `__init__` — os nomes das classes são livres
- Os cinco atributos presentes nas duas classes, com esses mesmos nomes
- `receber_dano` funcionando igual nas duas, com o dano nunca menor que 1

Exercício 2

MÉTODO ATACAR(ALVO)

Escrevam um método `atacar(self, alvo)`, onde `alvo` é *outro objeto* (Guerreiro, Mago, ou o que vocês estiverem usando) — não um número. Dentro do método, chamem `alvo.receber_dano(self.ataque)`, e imprimam uma mensagem dizendo quem atacou quem.

REQUISITOS MÍNIMOS

- `atacar` recebe outro objeto como parâmetro, além de `self`
- Chama o método `receber_dano` do **alvo**, não do próprio personagem
- Usa `self.ataque` como o valor do dano causado

Exercício 3

MÉTODO DEFENDER()

Adicionem um atributo `self.defendendo = False` no `__init__`. O método `defender()` muda esse valor pra `True` e avisa que o personagem está se defendendo. Depois, atualizem `receber_dano` pra reduzir o dano pela metade quando `self.defendendo` for `True` — e, ao final do `receber_dano`, voltar `defendendo` pra `False` de novo (a defesa vale só até o próximo ataque recebido, não pra sempre).

REQUISITOS MÍNIMOS

- `defendendo` começa como `False` no `__init__`
- `defender()` muda `defendendo` pra `True`
- `receber_dano` reduz o dano pela metade quando `defendendo` é `True`
- `defendendo` volta pra `False` depois de um ataque ser recebido

Exercício 4

POÇÕES: USAR_POCAO()

Método `usar_pocao()` que cura uma quantidade fixa de vida (por exemplo, 20) e reduz `self.pocoes` em 1 — mas só se ainda houver poções sobrando. Se `pocoes` for 0, avisem que não há mais poções, sem curar nada.

REQUISITOS MÍNIMOS

- Usa `if/else` pra checar se `self.pocoes > 0` antes de curar
- `vida` só aumenta quando existe pelo menos uma poção disponível
- `pocoes` nunca fica negativo

Exercício 5

UMA CENA DE TESTE · SEM LAÇO AINDA

Sem usar `while`, criem dois personagens das classes que vocês escolheram no Exercício 1 (sejam Guerreiro/Mago ou as suas próprias) e escrevam uma pequena cena manual, chamando os métodos na ordem que quiserem: alguns ataques, uma defesa, o uso de uma poção. Depois de cada ação, mostrem a vida dos dois personagens pra conferir se os números fazem sentido.

REQUISITOS MÍNIMOS

- Pelo menos 4 ações diferentes chamadas, misturando `atacar`, `defender` e `usar_pocao`
- A vida dos dois personagens é exibida depois de cada ação
- Nenhuma ação usa números soltos direto no `print` — tudo passa pelos métodos das classes

Próxima aula: transformar essa cena manual num laço de verdade, com `input()` e `random` — e, mais pra frente, começar a pensar em movimento.